

Εφαρμογή online παραγγελίας ποδιών (LogoFeet)

ΕΜΜΑΝΟΥΉΛ ΠΑΞΙΜΑΔΑΚΗΣ 1100972 & ΔΗΜΗΤΡΙΟΣ ΦΙΛΩΠΟΥΛΟΣ 1101130,

Τμήμα ΗΜ&ΤΥ, Πανεπιστήμιο Πατρών, Ελλάδα

Περίληψη: Η παρούσα εργασία παρουσιάζει την ανάπτυξη της web εφαρμογής LogoFeet στα πλαίσια του μαθήματος "Προγραμματισμός Διαδικτύου". Η εφαρμογή εξελίχθηκε από ένα αρχικό λειτουργικό πρωτότυπο (static HTML/CSS) σε μια πλήρη Full-Stack εφαρμογή που χρησιμοποιεί Node.js, Express και Handlebars, υποστηρίζοντας δυναμική διαχείριση δεδομένων και γεωχωρικές λειτουργίες.

1 Εισαγωγή

Σκοπός του project είναι η εξοικείωση με τις σύγχρονες τεχνολογίες διαδικτύου για τη δημιουργία μιας δυναμικής πλατφόρμας παραγγελιοληψίας. Η ανάπτυξη ακολούθησε μια σταδιακή προσέγγιση, ξεκινώντας από το σχεδιασμό της διεπαφής και καταλήγοντας στην υλοποίηση σύνθετης επιχειρησιακής λογικής στο backend.

2 Λειτουργικές Απαιτήσεις και Σενάρια Χρήσης

Το σύστημα σχεδιάστηκε για να καλύπτει πλήρως τον κύκλο ζωής μιας παραγγελίας, εξυπηρετώντας δύο διακριτούς ρόλους (Πελάτης και Εστιατόριο). Οι κύριες λειτουργικές απαιτήσεις περιλαμβάνουν:

2.1 Λειτουργίες Πελάτη

- Αναζήτηση & Φιλτράρισμα:** Δυνατότητα εύρεσης εστιατορίων που εξυπηρετούν τη συγκεκριμένη τοποθεσία (ακτίνα 4χλμ) μέσω ενσωμάτωσης χάρτη και φιλτραρίσματος βάσει κατηγοριών κουζίνας.
- Διαχείριση Προφίλ & Διευθύνσεων:** Εγγραφή, σύνδεση, και δυνατότητα αποθήκευσης πολλαπλών διευθύνσεων παράδοσης.
- Διαχείριση Καλαθιού & Checkout:** Προσθήκη προϊόντων, υπολογισμός συνόλου με μεταφορικά και ολοκλήρωση παραγγελίας, η οποία είναι διαθέσιμη ακόμα και για επισκέπτες (Guest Checkout).
- Live Tracking & Αξιολόγηση:** Παρακολούθηση της πορείας της παραγγελίας σε πραγματικό χρόνο και δυνατότητα αξιολόγησης του εστιατορίου μετά την ολοκλήρωση.

2.2 Λειτουργίες Εστιατορίου

- Διαχείριση Καταλόγου (Menu):** Προσθήκη, επεξεργασία, διαγραφή προϊόντων, εικόνων, τιμών και ένταξή τους σε κατηγορίες.
- Dashboard Παραγγελιών:** Πίνακας ελέγχου για την παραλαβή νέων παραγγελιών και την ενημέρωσή τους (π.χ., PREPARING, DELIVERING).
- Ρυθμίσεις Κατάστατος:** Καθορισμός ελάχιστης παραγγελίας και διαχείριση της συνολικής κατάστασης λειτουργίας.

3 Αρχιτεκτονική Model-View-Controller (MVC)

Για τη βέλτιστη οργάνωση, συντηρησιμότητα και επεκτασιμότητα του κώδικα, η αρχιτεκτονική της εφαρμογής δομήθηκε αυστηρά πάνω στο σχεδιαστικό πρότυπο **Model-View-Controller (MVC)**. Αυτός ο διαχωρισμός αρμοδιοτήτων επιτρέπει την ανεξάρτητη ανάπτυξη της διεπαφής χρήστη, της επιχειρησιακής λογικής και της διαχείρισης δεδομένων.

3.1 Επίπεδο Δεδομένων (Model)

Το επίπεδο **Model** (/model) είναι αποκλειστικά υπεύθυνο για την αλληλεπίδραση με τη βάση δεδομένων και την υλοποίηση των κανόνων της επιχειρησιακής λογικής (business logic). Στην υλοποίησή μας, δημιουργήθηκαν εξειδικευμένα αρχεία (π.χ., `auth-model.mjs`, `order-model.mjs`, `restaurant-model.mjs`) τα οποία εκτελούν παραμετροποιημένα ερωτήματα SQL (Parameterized Queries) μέσω του MySQL driver. Αυτή η αφαίρεση (abstraction) εξασφαλίζει ότι το υπόλοιπο σύστημα αγνοεί τις ιδιαιτερότητες της βάσης δεδομένων, ενώ ταυτόχρονα παρέχει προστασία από επιθέσεις SQL Injection.

3.2 Επίπεδο Παρουσίασης (View)

Το επίπεδο **View** (/views) αναλαμβάνει την οπτική αναπαράσταση των δεδομένων στον τελικό χρήστη. Αντί για στατικά HTML αρχεία, χρησιμοποιήθηκε η μηχανή δημιουργίας προτύπων (template engine) **Handlebars** (.hbs). Η προσέγγιση αυτή προσφέρει πολλαπλά πλεονεκτήματα:

- **Δυναμική Δημιουργία Περιεχομένου:** Επιτρέπει την ενσωμάτωση μεταβλητών, βρόχων (`{{#each}}`) και δομών ελέγχου (`{{#if}}`) απευθείας στο HTML markup.
- **Επαναχρησιμοποίηση Κώδικα:** Η χρήση ενός κεντρικού layout (`main.hbs`) και επιμέρους τμημάτων (partials) μειώνει τον πλεονάζοντα κώδικα και διευκολύνει τις καθολικές αλλαγές στο UI (π.χ. στο header ή το footer).

3.3 Επίπεδο Ελέγχου (Controller)

Το επίπεδο **Controller** (/controller) λειτουργεί ως ο ενδιάμεσος "τροχονόμος" (router) μεταξύ του χρήστη, του Model και του View. Κάθε ξεχωριστή οντότητα του συστήματος διαθέτει τον δικό της ελεγκτή (π.χ., `cart-controller.mjs`, `index-controller.mjs`). Οι Controllers είναι υπεύθυνοι για:

- (1) Την παραλαβή και επικύρωση (validation) των HTTP αιτημάτων (GET, POST, κ.λπ.) από τον Client.
- (2) Την κλήση των κατάλληλων μεθόδων του Model για ανάκτηση ή αποθήκευση δεδομένων.
- (3) Την προετοιμασία των δεδομένων (data formatting) και την προώθησή τους στο κατάλληλο View για τελική απόδοση (render) ή την εκτέλεση ανακατευθύνσεων (redirects).

4 Σχεδιασμός Βάσης Δεδομένων

Η βάση δεδομένων της εφαρμογής είναι σχεσιακή (RDBMS) και αποτελείται από πίνακες που απεικονίζουν τις βασικές οντότητες του συστήματος και τις μεταξύ τους σχέσεις. Τα βασικά δομικά στοιχεία είναι:

- **Users & Roles:** Διαχείριση λογαριασμών (Πελάτες, Εστιατόρια) με χρήση διακριτών ρόλων.
- **Address:** Αποθήκευση συντεταγμένων (Latitude/Longitude) και πλήρους κειμένου διεύθυνσης για την υποστήριξη των γεωχωρικών αναζητήσεων.
- **Menu & Categories:** Οργάνωση των διαθέσιμων προϊόντων κάθε εστιατορίου ανά κατηγορία.
- **Orders & Order Items:** Καταγραφή του ιστορικού παραγγελιών, με λεπτομέρειες για το κάθε προϊόν, τη διεύθυνση παράδοσης (μέσω snapshot) και το status της παραγγελίας.
- **Reviews:** Αποθήκευση βαθμολογιών για τον υπολογισμό του μέσου όρου (rating) κάθε εστιατορίου.

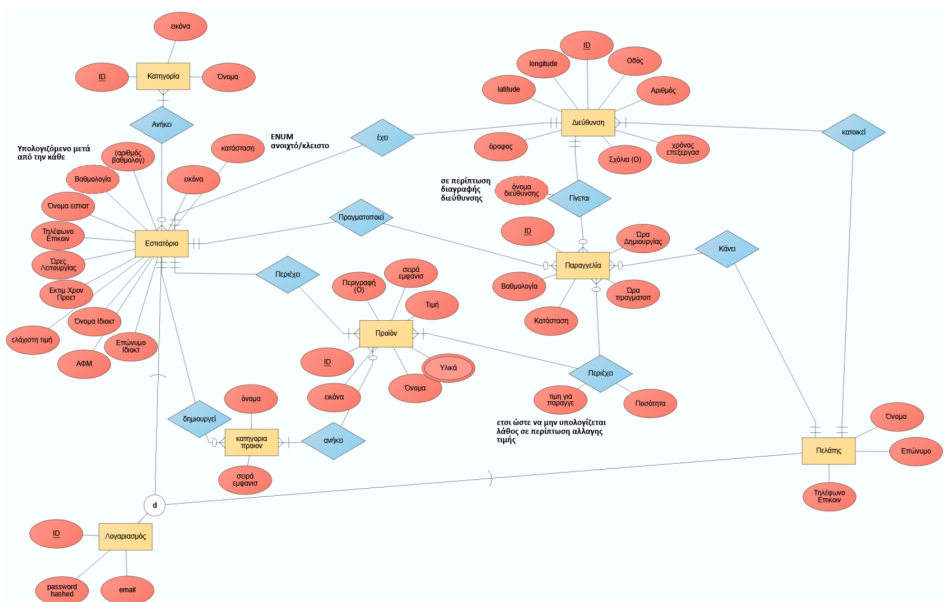


Fig. 1. Το διάγραμμα σχέσεων-οντοτήτων της βάσης δεδομένων.

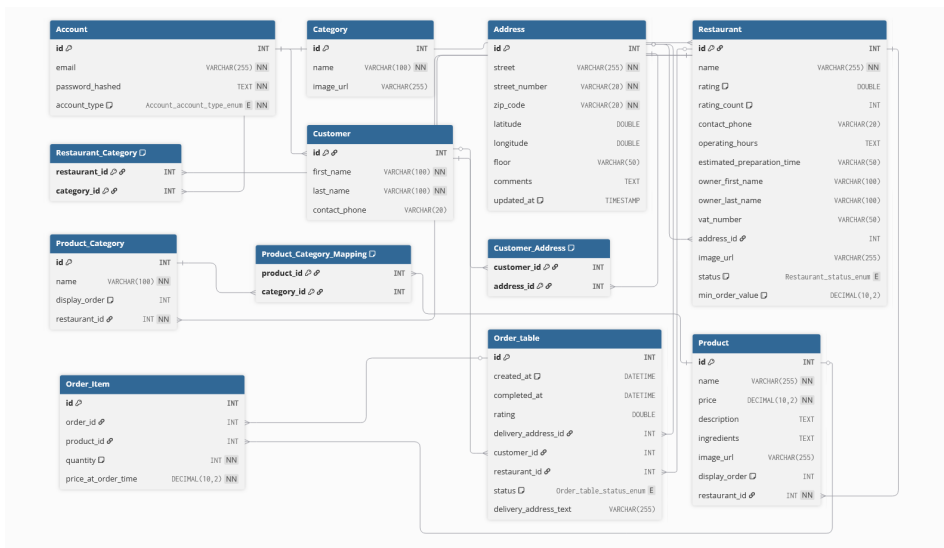


Fig. 2. Schema της βάσης δεδομένων.

5 Τεχνολογίες Υλοποίησης

5.1 Frontend (Διεπαφή Χρήστη)

Για τη διεπαφή επιλέχθηκαν τεχνολογίες που εξασφαλίζουν ταχύτητα και συμβατότητα:

- **Bootstrap 5 & Vanilla CSS:** Χρησιμοποιήθηκαν για τη δημιουργία ενός μοντέρνου, responsive σχεδιασμού (mobile-first approach). Η χρήση έτοιμων components της Bootstrap (cards, modals, alerts) επιτάχυνε σημαντικά την ανάπτυξη.
- **Handlebars (HBS):** Ως μηχανή προτύπων επέτρεψε την καθαρή ενσωμάτωση δυναμικών δεδομένων στα HTML αρχεία.
- **Leaflet.js & OpenStreetMap:** Αποτέλεσαν τη βάση για τη διαδραστική απεικόνιση χαρτών και την επιλογή τοποθεσιών, αποτελώντας μια ισχυρή, open-source εναλλακτική έναντι των κλειστών εμπορικών λύσεων.

5.2 Backend (Εξυπηρετητής)

Η "καρδιά" του συστήματος αναπτύχθηκε σε περιβάλλον JavaScript, επιτρέποντας πλήρη ομοιογένεια (Isomorphic JavaScript) μεταξύ Frontend και Backend:

- **Node.js & Express.js:** Το Node.js, χάρη στη μη-μπλοκαρισμένη I/O αρχιτεκτονική του (non-blocking), είναι ιδανικό για την ταυτόχρονη διαχείριση πολλαπλών requests. Το Express.js προσέφερε το απαραίτητο routing και middleware architecture.
- **Middlewares:** Υλοποιήθηκαν custom λύσεις (π.χ., έλεγχος πρόσβασης/authorization) και αξιοποιήθηκαν βιβλιοθήκες όπως το express-session για τη διαχείριση συνεδριών χρήστη.

5.3 Διαχείριση Δεδομένων και Μετάβαση

Στο πρώτο στάδιο ανάπτυξης χρησιμοποιήθηκε η SQLite3 για την ταχεία δημιουργία πρωτοτύπου. Στην τελική έκδοση, η εφαρμογή μεταφέρθηκε σε cloud περιβάλλον (TiDB Cloud) με χρήση MySQL, διασφαλίζοντας κλιμακωσιμότητα και απομακρυσμένη πρόσβαση.

6 Geospatial Λειτουργίες και Περιορισμοί Απόστασης

Ένα από τα κρίσιμότερα χαρακτηριστικά της εφαρμογής είναι ο περιορισμός της εμβέλειας παράδοσης των εστιατορίων στα 4 χιλιόμετρα. Για την υλοποίηση αυτής της απαίτησης, αναπτύχθηκε ένα σύστημα που συνδυάζει client-side γεωκωδικοποίηση και server-side επαλήθευση.

6.1 Υπολογισμός Απόστασης με τον Τύπο Haversine

Για τον ακριβή υπολογισμό της απόστασης μεταξύ του πελάτη και του εστιατορίου στην επιφάνεια της γης, χρησιμοποιήθηκε ο μαθηματικός τύπος **Haversine**. Ο τύπος αυτός λαμβάνει υπόψη την καμπυλότητα της γης και υπολογίζεται ως εξής:

$$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right) \quad (1)$$

$$c = 2 \cdot \arctan 2(\sqrt{a}, \sqrt{1-a}) \quad (2)$$

$$d = R \cdot c \quad (3)$$

όπου ϕ είναι το γεωγραφικό πλάτος (latitude), λ το γεωγραφικό μήκος (longitude) σε ακτίνια (radians), και $R = 6371$ km η μέση ακτίνα της Γης.

Η συνάρτηση `getDistanceKm` υλοποιήθηκε στο αρχείο `geo-utils.mjs` και καλείται κατά τη διαδικασία του checkout για να διασφαλίσει ότι η παραγγελία δεν υπερβαίνει το όριο των 4 χιλιομέτρων.

6.2 Αυτόματο Geocoding

Για τη διευκόλυνση του χρήστη, ενσωματώθηκε το API της υπηρεσίας OpenStreetMap (Nominatim). Όταν ο χρήστης πληκτρολογεί τη διεύθυνσή του, το σύστημα εκτελεί ασύγχρονα requests (AJAX) για την εύρεση των συντεταγμένων.

Κατά την υλοποίηση παρατηρήθηκε ότι αν ο χρήστης άλλαζε τη διεύθυνση χειροκίνητα στα πεδία κειμένου μετά από μια επιλογή στο χάρτη, οι κρυφές συντεταγμένες παρέμεναν οι παλιές. Για την επίλυση αυτού του προβλήματος, προστέθηκαν event listeners που μηδενίζουν τις συντεταγμένες μόλις ο χρήστης τροποποιήσει τα πεδία της διεύθυνσης, αναγκάζοντας το σύστημα να εκτελέσει νέα γεωκωδικοποίηση κατά την υποβολή της φόρμας.

7 Παρακολούθηση Παραγγελιών σε Πραγματικό Χρόνο (Live Tracking)

Για την αναβάθμιση της εμπειρίας του χρήστη, υλοποιήθηκε ένα σύστημα παρακολούθησης της εξέλιξης της παραγγελίας (Live Tracking). Ο πελάτης μπορεί να βλέπει σε ποιο στάδιο βρίσκεται η παραγγελία του σε πραγματικό χρόνο.

7.1 Μηχανή Καταστάσεων Παραγγελίας

Η παραγγελία διέρχεται από τις εξής διακριτές καταστάσεις (States), οι οποίες ενημερώνονται από το εστιατόριο μέσω του δικού του Dashboard:

- **PENDING (10%):** Η παραγγελία έχει υποβληθεί και εκκρεμεί η αποδοχή της.
- **PREPARING (35%):** Το εστιατόριο προετοιμάζει τα προϊόντα.
- **READY (60%):** Τα προϊόντα είναι έτοιμα προς παράδοση.
- **DELIVERING (85%):** Η παραγγελία βρίσκεται καθ' οδόν προς τον πελάτη.
- **COMPLETED (100%):** Η παραγγελία παραδόθηκε επιτυχώς.

Στο frontend (track-orders.hbs), χρησιμοποιήθηκε μια μπάρα προόδου (Progress Bar) της Bootstrap, το πλάτος της οποίας διαμορφώνεται δυναμικά βάσει της κατάστασης της παραγγελίας, προσφέροντας μια συνεχή και κατανοητή οπτική ένδειξη στον χρήστη.

8 Ανάπτυξη και Εξέλιξη (Project Timeline)

Η ανάπτυξη της εφαρμογής βασίστηκε στο σύστημα ελέγχου εκδόσεων Git. Οι κυριότεροι σταθμοί στην εξέλιξη του project, όπως προκύπτουν από το ιστορικό των commits, είναι:

- **Έναρξη και Πρωτοτυποποίηση:** Σχεδιασμός των πρώτων στατικών σελίδων και υλοποίηση της αναζήτησης τοποθεσίας.
- **Μετάβαση σε Full-Stack (Express/Handlebars):** Μετατροπή του στατικού site σε δυναμική εφαρμογή με τη χρήση του Node.js και της μηχανής Handlebars.
- **Ενσωμάτωση Βάσης Δεδομένων:** Σύνδεση με SQLite αρχικά και μετέπειτα μετάβαση σε cloud υποδομή (TiDB Cloud / MySQL).
- **Σύστημα Παρακολούθησης (Live Tracking):** Προσθήκη της σελίδας παρακολούθησης και της μπάρας προόδου.
- **Διαχείριση και Ασφάλεια:** Προσθήκη του Restaurant Dashboard, κρυπτογράφηση κωδικών και προστασία δεδομένων.
- **Geospatial Λειτουργίες:** Υλοποίηση του υπολογισμού αποστάσεων και εκτίμησης χρόνου παράδοσης.

9 Ασφάλεια

- Χρήση περιβάλλοντος .env για την προστασία των API keys και των credentials της βάσης.
- Κρυπτογράφηση κωδικών πρόσβασης με τη βιβλιοθήκη Bcrypt.

- Προστασία από SQL Injection μέσω του MySQL driver (placeholders).

10 Τεκμηρίωση API και Δρομολόγηση (Routes)

Για την καλύτερη κατανόηση και τεκμηρίωση των λειτουργιών του συστήματος, δημιουργήθηκε ένα διαδραστικό περιβάλλον τεκμηρίωσης μέσω της βιβλιοθήκης flask-restx σε περιβάλλον Python (Swagger UI). Τα δρομολόγια της εφαρμογής χωρίζονται στις εξής βασικές κατηγορίες:

10.1 Δημόσια Δρομολόγια και Αυθεντικοποίηση

- GET /: Αρχική σελίδα (Landing Page).
- GET /browse: Αναζήτηση και περιήγηση εστιατορίων.
- GET/POST /login: Σύνδεση χρηστών και εστιατορίων.
- GET/POST /register: Εγγραφή νέου πελάτη.
- GET/POST /register-restaurant/step1 & step2: Διαδικασία εγγραφής εστιατορίου σε δύο στάδια.

10.2 Διαχείριση Καλαθιού και Παραγγελιών

- GET /cart: Προβολή του καλαθιού αγορών.
- POST /cart/add: Προσθήκη προϊόντος στο καλάθι.
- POST /cart/checkout: Ολοκλήρωση παραγγελίας.
- GET /track-orders: Παρακολούθηση της πορείας των παραγγελιών.

10.3 Διαχείριση Εστιατορίου

- GET /manage/orders: Διαχείριση εισερχόμενων παραγγελιών.
- POST /manage/products: Προσθήκη νέου προϊόντος στον κατάλογο.
- POST /manage/status: Εναλλαγή κατάστασης λειτουργίας (Ανοιχτό/Κλειστό).

Η χρήση του Swagger UI επιτρέπει στους προγραμματιστές να δοκιμάζουν τα endpoints και να βλέπουν τα αναμενόμενα σχήματα δεδομένων (Schemas) για κάθε αίτημα.

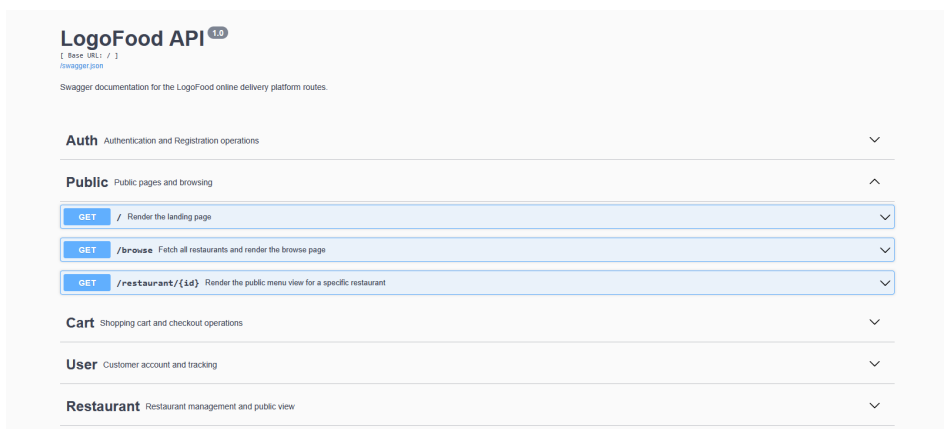


Fig. 3. Απεικόνιση των routes της εφαρμογής στο Swagger UI.

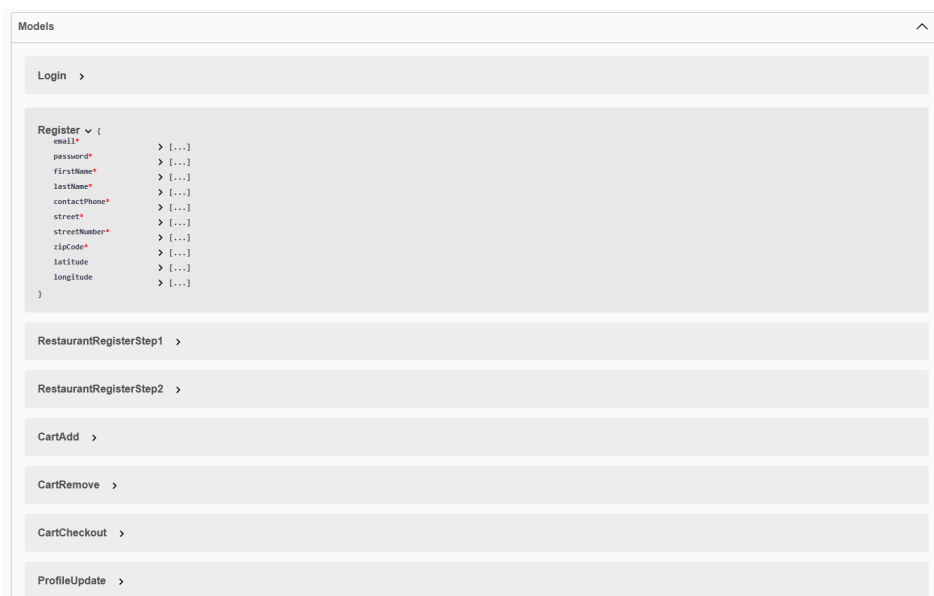


Fig. 4. Μοντέλα δεδομένων του API.

11 Προβλήματα και Λύσεις που Εμφανίστηκαν

Κατά την ανάπτυξη του συστήματος παραγγελιών, αντιμετωπίστηκαν προκλήσεις που αφορούσαν τη διατήρηση του ιστορικού των παραγγελιών σε σχέση με τις διαγραφές δεδομένων από τους χρήστες.

11.1 Το Πρόβλημα της Διαγραφής Διευθύνσεων

Στο αρχικό σχήμα της βάσης δεδομένων, ο πίνακας `Order_table` συνδεόταν με τον πίνακα `Address` μέσω ξένου κλειδιού. Η διαγραφή μιας διεύθυνσης από το προφίλ του χρήστη οδηγούσε στον μηδενισμό του πεδίου `delivery_address_id` (λόγω του κανόνα `ON DELETE SET NULL`).

Αυτό προκάλεσε το φαινόμενο οι παραγγελίες να "εξαφανίζονται" από το ιστορικό του χρήστη, καθώς οι ερωτήσεις στη βάση χρησιμοποιούσαν `INNER JOIN`. Το πρόβλημα λύθηκε με τη μετάβαση σε `LEFT JOIN`, επιτρέποντας την εμφάνιση των παραγγελιών ακόμα και αν η διεύθυνση είχε διαγραφεί.

11.2 Διατήρηση Ιστορικού με Στιγμιότυπα (Snapshotting)

Παρά τη λύση του `LEFT JOIN`, αν η διεύθυνση είχε διαγραφεί, το σύστημα δεν μπορούσε να εμφανίσει πού είχε παραδοθεί η παραγγελία. Για την επίλυση αυτού, εφαρμόστηκε η τεχνική του **Snapshotting**.

Προστέθηκε η στήλη `delivery_address_text` στον πίνακα `Order_table`. Κατά τη δημιουργία της παραγγελίας, η πλήρης διεύθυνση αποθηκεύεται ως κείμενο απευθείας στην παραγγελία. Έτσι, ακόμα και αν ο χρήστης διαγράψει ή τροποποιήσει τη διεύθυνσή του στο μέλλον, το ιστορικό της παραγγελίας παραμένει αναλλοίωτο και ακριβές.

12 Συμπεράσματα

Η ενασχόληση με το project LogoFeet προσέφερε πολύτιμη εμπειρία σε όλο το φάσμα της ανάπτυξης διαδικτυακών εφαρμογών, από το σχεδιασμό της εμπειρίας χρήστη (UX) μέχρι την επίλυση σύνθετων προβλημάτων στο backend και τη διαχείριση cloud υποδομών.

13 Οδηγίες Εγκατάστασης

- (1) `npm install` (εγκατάσταση εξαρτήσεων Node.js).
- (2) Αντιγραφή του `.env` για ρύθμιση της σύνδεσης TiDB.
- (3) `npm run dev` για εκκίνηση του development server.

A Στιγμιότυπα Εφαρμογής

Fig. 5. Αρχική σελίδα με ενσωμάτωση χάρτη Leaflet.